

Föreläsning 3

Programutvecklingsteknik

Introduktion till designmönster

Singleton

MVC

Innehåll

- `static`
- Åtkomstmodifierare
- UML
 - Association
 - Dependency (beroende)
- Introduktion till designmönster
 - Singleton
 - Model-View-Controller
- Mock-objekt

`static` för variabler och metoder (inte klasser)

När en variabel eller metod deklarerar med **`static`** tillhör variabeln eller metoden **klassen** i stället för ett specifikt objekt. Det innebär att **alla instanser av klassen delar på samma variabel eller metod**. Det passar därför när man vill lagra information som ska vara gemensam för alla objekt av klassen.

static för variabler och metoder (inte klasser)

Koden i Main visar hur värden av variabeln `kvar` som är klassvariabel och variabeln `ivar` som är en instansvariabel ändras.

```
class A{
    static int kvar;
    int ivar;
    A(int k, int i){
        this.ivar = i;
        A.kvar = k;
    }
}
```

```
class Main{
    public static void main(String[] args){
        A ett = new A(5,7);
        System.out.println(ett.kvar+"_"+ett.ivar); //5_7
        A tva = new A(50,70);
        System.out.println(ett.kvar + "_" + ett.ivar); // 50_7
        System.out.println(tva.kvar + "_" + tva.ivar); // 50_70
        System.out.println(A.kvar); //50
    }
}
```

static för variabler och metoder (inte klasser)

- när en variabel deklarerar med `static`, det innebär att variabeln tillhör klassen (samt ALLA instanser av klassen).

```
class A{
    static int kvar;
    int ivar;
    A(int k, int i){
        this.ivar = i;
        A.kvar = k;
    }
}
```

```
class Main{
    public static void main(String[] args){
        A ett = new A(5,7);
        System.out.println(ett.kvar+"_"+ett.ivar); //5_7
        A tva = new A(50,70);
        System.out.println(ett.kvar + "_" + ett.ivar); // 50_7
        System.out.println(tva.kvar + "_" + tva.ivar); // 50_70
        System.out.println(A.kvar); //50
    }
}
```

static för variabler och metoder (inte klasser)

En variabel eller en metod som deklarerats med `static` kallas klassvariabel och klassmetod

```
class A{
    static int kvar;
    int ivar;
    A(int k, int i){
        this.ivar = i;
        A.kvar = k;
    }
    static int m(){
        kvar = 12;
        ivar = 24; //FEL
    }
}
```

```
class Main{
    public static void main(String[] args){
        A ett = new A(5,7);
        System.out.println(ett.kvar+"_"+ett.ivar); //5_7
        A tva = new A(50,70);
        System.out.println(ett.kvar + "_" + ett.ivar); // 50_7
        System.out.println(tva.kvar + "_" + tva.ivar); // 50_70
        System.out.println(A.kvar); //50
    }
}
```

Åtkomstmodifierare

- `public`: åtkomlig för alla klasser oavsett
- `private`: åtkomlig endast i samma klass
- (default eller `package-private`): åtkomlig endast inom samma paket
- `protected`: åtkomlig inom samma paket + alla subklasser

```
public class Person {  
    public String name = "Anna";  
}
```

```
Person p = new Person();  
System.out.println(p.name); // Fungerar från alla klasser och paket
```

```
public class Person {  
    private int age = 30;  
}
```

```
Person p = new Person();  
System.out.println(p.age); // FEL: age är private  
System.out.println(p.getAge()); // OK via getter
```

```
public class Car {  
    String model = "Volvo"; // default (package-private)  
}
```

```
Car c = new Car();  
System.out.println(c.model); // Fungerar i samma paket, men inte utanför det
```

```
public class Animal{  
    protected String type = "Dog";  
}
```

```
public class Dog extends Animal{  
    public void printType() {  
        System.out.println(type); // OK (subklass får åtkomst)  
    }  
}
```

```
Dog d = new Dog();
```

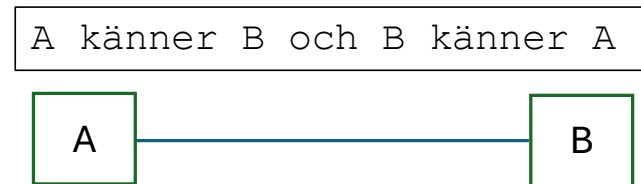
```
d.printType(); // Skriver ut "Dog"
```

```
System.out.println(d.type); fungerar bara i samma paket
```

UML

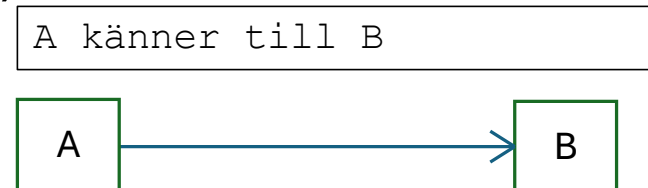
- Association

```
class A{  
    B refB;  
...  
}  
class B{  
    A refA;  
...  
}
```



- Directed association (riktad association)

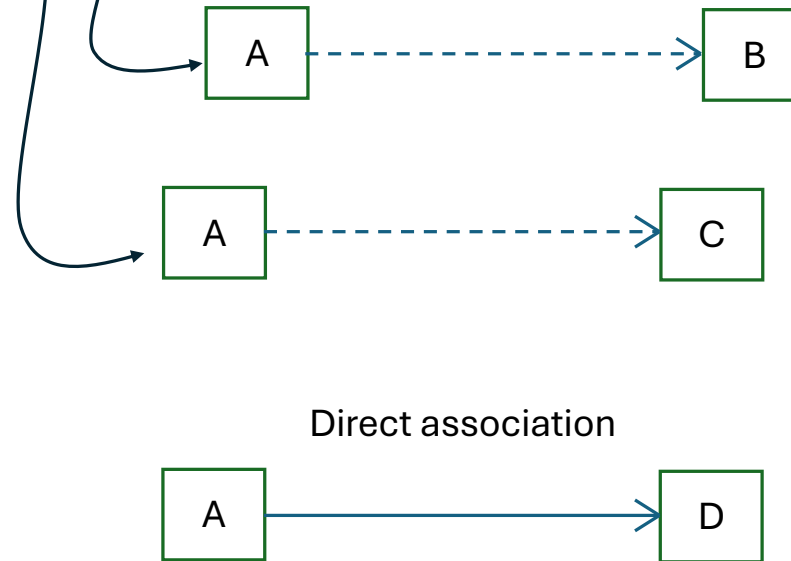
```
class A{  
    B refB;  
...  
}  
class B{...  
}
```



UML

- Dependency (beroende)

```
class A{  
    D refD;  
    void enMetod(B refB) {...}  
  
    void enMetod(){  
        C refC = new C();  
        ...  
    }  
    void enMetod(D refD){  
        this.refD = refD;  
    }  
}
```



Introduktion till designmönster

- Återanvändbara lösningar på problem som förekommer ofta inom programvarudesign
- Välja de beprövade mallar istället att uppfinna egna lösningar
- Höjer abstraktionen på två sätt:
 - Konceptuell språk, abstrakt nivå i kommunikation mellan utvecklare
 - Designmönstren är oftast baserat interface och abstrakta klasser
- Hjälper att undvika vanliga designfel då designmönstren tar hänsyn till designprinciperna SOLID

Designprinciper: **S**OLID

- **Single responsibility:** en klass ska ha ett och endast ett ansvar
- **Open/Closed:** En modul/klass ska vara öppen för utbyggnad men stäng för ändring. Ny funktionalitet via arv (open) och **INTE** genom att ändra i befintlig kod (Close)
- *Liskov substitution: Objekt av subbklass ska kunna användas som objekt av basklassen utan att bryta mot programmets logik*
- *Interface segregation: Klass ska inte tvingas implementera metoder de inte behöver. Fler små interface är bättre än en stor allmän.*
- *Dependency inversion: Arbeta mot abstraktioner (interface och abstrakta klasser) istället för konkreta implementationer.*

Singleton

Önskemål/krav:

Vi behöver definiera en klass, t.ex S, som ska användas av andra utvecklare som jobbar med samma projekt. Endast ETT objekt av den klassen ska kunna skapas.

När behövs det? Bl.a:

- Kontrollobjekt: en del saker behöver kontrolleras innan något får hända.
- Centralsystem, t.ex centralregistrering för att hantera tjänster och resurser
- Resurshantering (t.ex förhindrar undvika dubbelbokningar)

Varför fungerar följande kod?

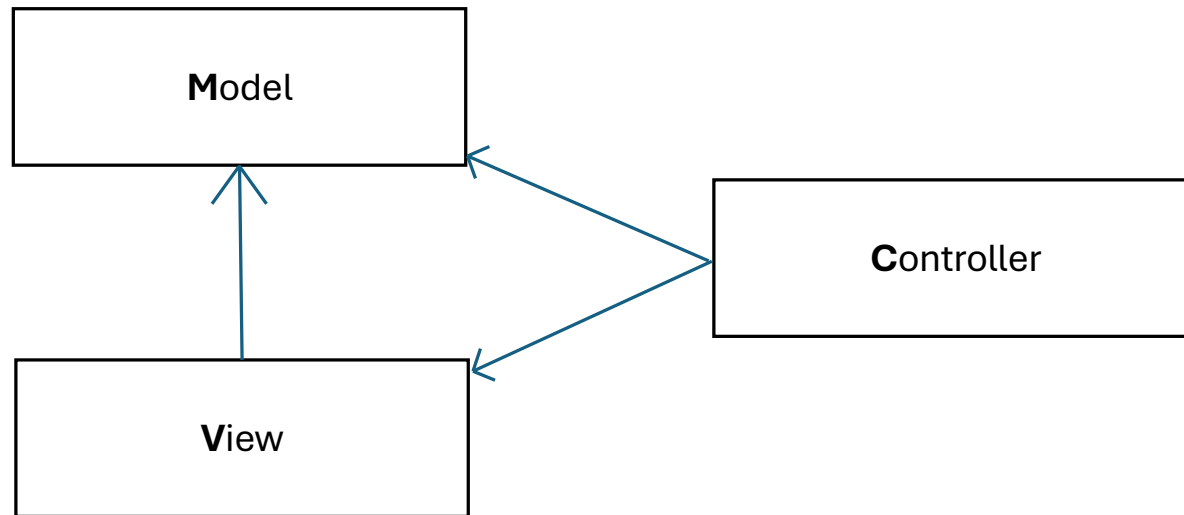
```
class Singleton{  
    private static Singleton s;  
  
    private Singleton(){}  
  
    public static Singleton getIntstanceOf(){  
        if (s == null)  
            s = new Singleton();  
        return s;  
    }  
}
```

Model-View-Controller

Dela ansvar mellan klasser

- Model
 - Representerar data och affärslogik
 - Känner varken View eller Controller
 - Ansvar för lagring och uppdatering av information
- View
 - Presentera data för användare
 - Känner till Model men inte Controller
 - Uppdateras när Model ändras
- Controller
 - Hanterar användar-intraktion (input)
 - Känner till både Model och View
 - Använder Model för att uppdatera data och View för att uppdatera presentation

MVC



Mock

En klass som tillhandhåller strukturen men stödjer inte funktionaliteten.
Använda för att utvecklare ska inte vara beroende på varandras arbete under utvecklingsfasen.